

Intent Driven Autonomous ORAN xApp Development using Agentic AI

Final Presentation

Mohit Patil (CS24MTECH14013)
Harshwardhan Gaikwad (CS25MTECH12004)

Project Recap (Brief)

- Problem statement

ORAN xApp development is complex and manual.
- Motivation
 - Building a tool that automates the xApp development saves time
- Key Objectives
 - Reproduce the AutORAN Paper from scratch
 - Build end to end pipeline that generates working xApps based on the Natural language intent.
 - Use a Module based approach to improve scalability and the system codebase agnostic

Related Work / Positioning

- List 2–3 closest works/systems and what they do -

1. Intent-Driven Network Management with Multi-Agent LLMs, SIGCOMM' 25

This paper presents Confucius, a production-ready multi-agent LLM framework that enhances large-scale network management through DAG-based workflows, RAG memory, tool integration, and strong validation mechanisms.

2. AutORAN: LLM-driven Natural Language Programming for Agile xApp Development

AutORAN is an LLM-driven framework that converts high-level user intents into fully deployable O-RAN xApps within minutes, automating the entire development pipeline and significantly accelerating network innovation.

- What gap your project addresses or reproduces?

The project reproduces core feature of the AutORAN paper by enabling modular, blueprint-driven and FlexRIC-grounded generation of deployable xApps from intent, improving reliability and end-to-end practicality beyond prior LLM-based systems

System / Key Assumptions

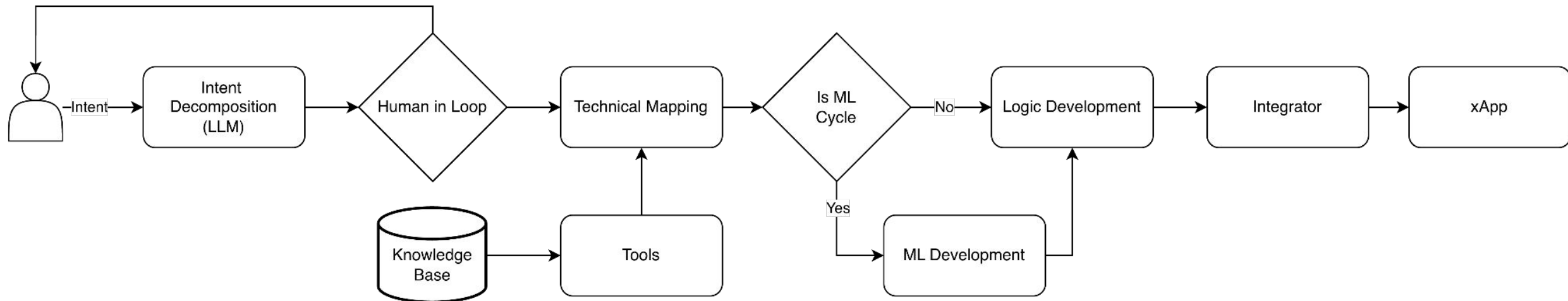
- System

- Multi-agent pipeline (intent decomposition → technical mapping → data/ML → logic → integration)
- LLM agents orchestrated via LangGraph
- Shared structured blueprint state across modules
- Semantic and Structural RAG system for FlexRIC code grounding
- Workspace tools for code generation and execution
- Optional ML training + data pipeline (synthetic or real datasets)
- Final FlexRIC xApp is delivered at the end of the pipeline

- Key Assumptions

- LLM outputs are reliable when constrained by structured prompts and RAG grounding.
- The user provides a coherent, actionable network management intent

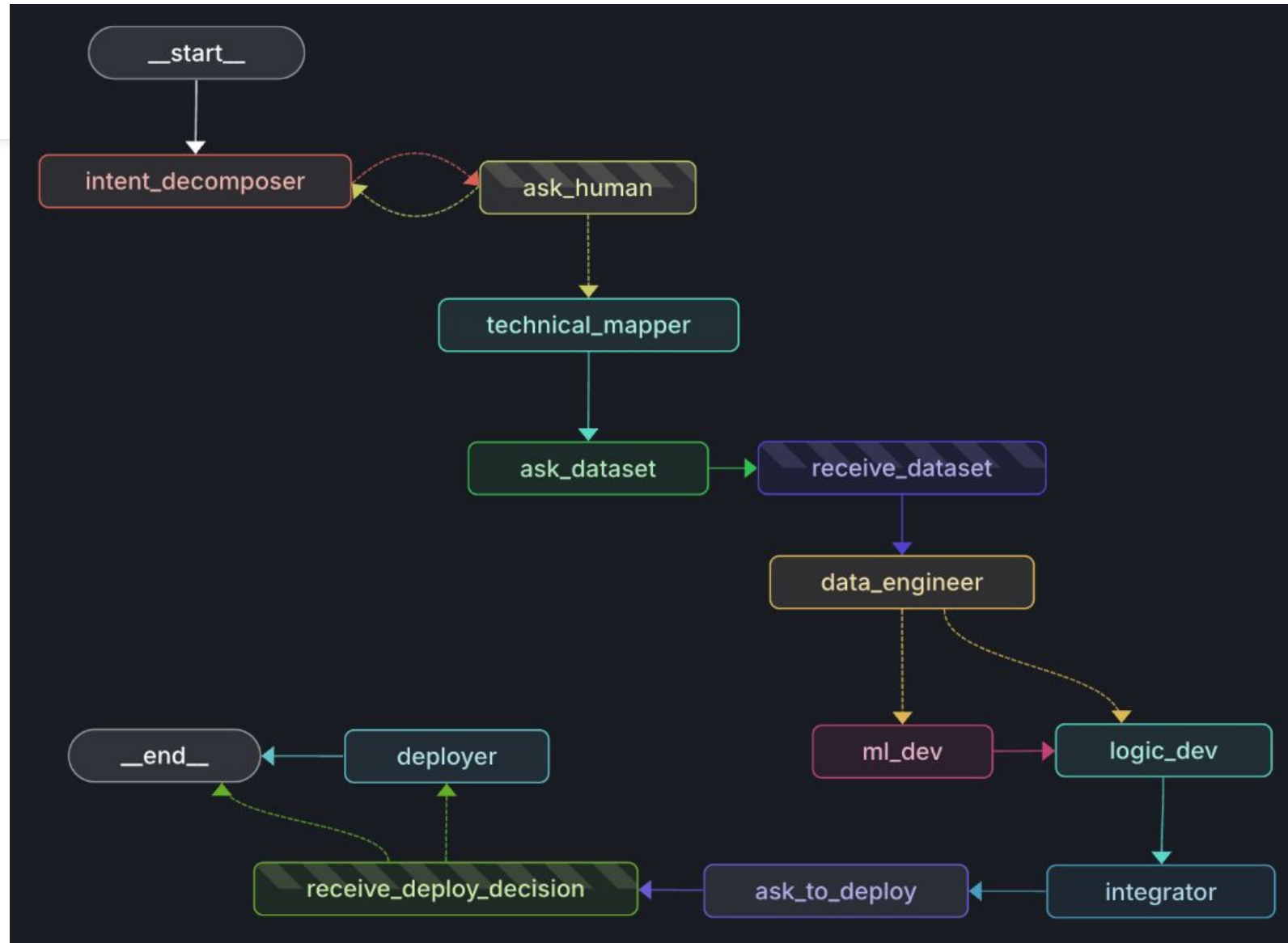
System Architecture/Methodology (1-2 slides)



Key Contributions / Highlights:

- Blueprint-driven architecture: A shared structured state ensures consistency, traceability, and reduced hallucination across agents.
- End-to-end automation: Covers the full lifecycle from natural-language intent to deployable xApp (not just code generation).
- FlexRIC-grounded RAG: Uses Semantic and Structural RAG over real codebases for accurate telemetry/action mapping.
- Human-in-the-loop checkpoints: Improves correctness at critical stages (intent + dataset decisions).
- Dual data pathway: Supports both synthetic data generation and real dataset ingestion seamlessly.
- Modular agent design: Clear separation of concerns improves reliability, debugging, and extensibility

Flowchart



Implementation Details

- Hardware Setup (if applicable):
 - Local System, Lab Server with GPUs for local deployment
- Software & Tools:

LangChain (for Agent orchestration), Ollama/OpenAI (LLM backbone), FAISS/ChromaDB for Dual-RAG indexing, Python, Docker, FlexRIC, OAI
- Implementation:

<https://github.com/mohitpatil30501/Intent-Driven-Autonomous-ORAN-xApp-Development-using-Agent-AI>

Implementation Details

What you built vs reused?

Reused / Extended

- LangChain / LangGraph: Used for orchestration, agent execution, and frontend (LangSmith + LangChain UI).
- OrioSearch: Reused as a web-search fallback and extended for O-RAN-specific queries.
- Semantic Search service: Reused and extended, later complemented/replaced by a more structured RAG approach, context length pre.

Built from scratch

- Modules 1–6: Complete end-to-end agent pipeline (intent decomposition → deployment).
- FlexRIC Structural RAG: Custom code-aware retrieval system for accurate API/telemetry grounding.
- Blueprint-driven architecture: Shared state design across modules.
- Terminal Command Runner: Extended LangChain file manager to safely execute generated code in a controlled workspace.

Implementation Details

Key challenges faced and how did you address them?

- Hallucinations in technical mapping → Mitigated using Structural RAG grounded in real FlexRIC code, instead of relying on pure LLM reasoning.
- Mismatch between natural language and system-level variables → Introduced a two-step mapping (intent → blueprint → technical mapping) to progressively constrain outputs.
- Unreliable data generation / schema alignment → Standardized Data_Paths contracts and enforced schema validation across modules.
- Context explosion and tool-calling loops → Added human-in-the-loop checkpoints and limit to tool call logic to limit unnecessary iterations.
- Integration complexity with FlexRIC → Separated core logic (Module 5) from deployment integration (Module 6) to isolate errors and simplify debugging.
- Ensuring end-to-end reliability → Designed a modular pipeline with clear contracts, allowing failures to be traced and fixed at specific stages instead of the whole system.

Experimental Setup

- Setup

- Hybrid deployment: remote GPU + local lab setup
- Ollama runs on remote GPU server
- LangGraph runs locally
- LLM inference routed from local system → GPU backend
- Tools, codebase, and FlexRIC Structural RAG run locally
- LangChain frontend for user interaction
- Backend orchestrates modules, tools, and data flow

- Reproducibility details

- Modular, reproducible setup with clear separation of components
- Ollama for GPU-backed inference
- LangGraph for local orchestration
- Local retrieval and tools via Structural RAG + workspace
- Environment variables manage endpoints and model configs
- Easily replicable across similar setups
- Codebase, README, and Quickstart available on GitHub

Experimental Setup

- Metrics used for Analysis

- API Fidelity, Build Success, E2SM Subscription, Intent Coverage
- ML Suitability (where applicable), ML Artifact Readiness (where applicable)
- Structural Soundness, Hallucination Inverse, Generation Latency, Readability

- Baseline(s) for Comparison

- Gemini 3.1 (standalone LLM) without RAG or agentic orchestration
- Gemma 4: e4B, evaluated as a smaller model baseline without system-level enhancements

Results & Analysis

Intent	Max	Gemma (e4b/8b)	Gemini 3.1 Fast	AutORAN
SliceDetailMonitor	40	9 (22.5%)	11 (27.5%)	35 (87.5%)
PRBMonitor	40	9 (22.5%)	11 (27.5%)	25 (62.5%)
UEClassifier	50	15 (30%)	11 (22%)	39 (78%)
UETrafficLoadBalancer	40	9 (22.5%)	11 (27.5%)	27 (67.5%)
AnomalyDetection	50	17 (34%)	14 (28%)	39 (78%)
Aggregate	220	59 (26.8%)	58 (26.4%)	165 (75.0%)

Results & Analysis (Per Criterion normalized Score)

Criterion	Gemma	Gemini	AutORAN
C1 — API Fidelity	0.0	0.0	3.4
C2 — Build Success	0.0	0.0	3.4
C3 — E2SM Subscription	0.0	0.0	4.6
C4 — Intent Coverage	3.0	3.0	4.2
C5 — ML Suitability (where applicable)	4.0	1.5	5.0
C6 — ML Artifact Readiness (where applicable)	3.0	0.0	5.0
C7 — Structural Soundness	0.0	0.0	5.0
C9 — Hallucination Inverse	0.0	0.0	3.4
C10 — Generation Latency	3.0	5.0	0.0
C11/C12 — Readability	3.0	3.0	5.0

Result Analysis

1. AutORAN dominates on platform-correctness criteria. It is the only system scoring non-zero on the FlexRIC-binding criteria (C1, C2, C3, C7, C9). Gemma and Gemini both flatline at 0 on every platform integration metric — neither produces deployable xApp code against FlexRIC.
2. Gemma and Gemini fail in the same way for different reasons. Gemma never tries to integrate (pure mock simulations); Gemini tries but hallucinates the wrong framework (ricxappframe from OSC, not FlexRIC's xapp_sdk). Their aggregate scores are nearly identical (~27%) — both essentially produce non-functional xApps.
3. AutORAN's weakness is C-struct field hallucination, not architecture. On 4 of 5 intents it loses points on C1/C2/C9 due to ~1–2 hallucinated struct properties per 100 LOC (e.g., kpi_indication_message instead of ue_stats, fabricated control fields). These are described as "≤3 trivial fixes" — i.e., near-deployable rather than fundamentally broken.
4. AutORAN has one substantive logic bug. The UETrafficLoadBalancer drops to 67.5% because core_logic.py hardcodes an 85% threshold (vs. the 80% the intent specified) and ignores the UE-count clause entirely. This is the only case where it misreads the explicit intent rather than just mis-binding C structs.
5. The latency tradeoff (C10) is real. AutORAN scores 0 on generation latency across the board, while Gemini gets a perfect 5. AutORAN is paying multi-agent orchestration cost to gain ~50 percentage points of correctness.
6. ML lifecycle handling is the clearest qualitative gap. On the two ML intents, AutORAN is the only system that treats models as pre-trained external artifacts with graceful fallbacks. Gemini retrains inside `__init__` on every launch (and even refits unsupervised models per micro-batch in AnomalyDetection), which is architecturally wrong for edge inference.

Limitations / Failure Cases

- Where does system break?

- Struggles with moderate–complex intents involving multiple telemetry metrics and interdependent actions.
- Errors in mapping C-based structs and extracting exact telemetry fields during code/RAG search.
- Occasional hallucination of non-existent technical mappings when converting NLP → technical mapping.
- Ambiguous intents can trap the LLM in repeated tool-calling loops, inflating context and degrading performance.
- Failures in data synthesis, especially when aligning generated dataset columns with actual telemetry variable names.
- Unexpected or redundant tool calls, leading to inefficiency and unstable execution flow.

- Unrealistic assumptions

Assuming NLP → structured intent → technical mapping is error-free is unrealistic; ambiguity and missing context frequently affect accuracy

- Scalability issues?

High latency and cost due to multiple sequential LLM calls across modules.

Context growth from multi-step reasoning and tool interactions can exceed limits or reduce model effectiveness.

Comparison with Objectives

Objective 1	Status (Completed/Partially Completed, Infeasible, etc)
Reproduce the AutORAN Paper from scratch	Completed
Use a Module based approach to improve scalability and the system codebase agnostic	Completed
Build end to end pipeline that generates working xApps based on the Natural language intent.	Partially Completed (Edge Case Exists)
Beat Spotlight Results	Infeasible with opensource models

Screenshots

The screenshot displays the Studio interface, which is divided into two main panels. The left panel shows a workflow graph with the following nodes and connections:

- intent_decomposer** (red box) connects to **ask_human** (yellow box).
- ask_human** connects to **technical_mapper** (green box).
- technical_mapper** connects to **ask_dataset** (green box).
- ask_dataset** connects to **receive_dataset** (purple box).
- receive_dataset** connects to **data_engineer** (yellow box).
- data_engineer** connects to **ml_dev** (pink box) and **logic_dev** (teal box).
- ml_dev** and **logic_dev** both connect to **integrator** (teal box).

The right panel shows a chat window titled "Thread resource_scheduler_optimizer". It displays a message from the AI agent, which is a JSON object:

```
json
{
  "Intent_Blueprint": {
    "validation": {
      "isComplete": true,
      "missingFields": [],
      "questionsForHuman": []
    },
    "xApp_Name": "resource_scheduler_optimizer",
    "code_language": "Python3",
    "objective_Why": "To achieve smarter resource distribution among connected UEs based on predicted load.",
    "target_Action_What_NL": "Modify PRB allocation (boosting or reducing resources) for specific UEs.",
    "cycle_Type": "Supervised_ML",
    "Service_Models": [
      "MAC"
    ],
    "requested_Telemetry_NL": [
      "Per-UE downlink throughput",
      "Per-UE uplink throughput"
    ],
    "data_Requirements": {
      "needs_historical_training_data": true,
      "historical_data_description_NL": "Each row contains an observation for a UE, featuring aggregated downlink and uplink throughput bytes (dl_aggr_tbs and ul_aggr_tbs) and a corresponding binary label (high_load).",
      "streaming_mock_data_description_NL": "Real-time stream mirrors historical format: two columns representing aggregated downlink and uplink throughput bytes, with no extra fields."
    },
    "model_acceptance_criteria": {
      "threshold": 0.85,
      "metric_policy": "task_aware",
      "metric_description_NL": "The model's ability to correctly distinguish between high-load and low-load UEs."
    }
  }
}
```

At the bottom of the interface, there is an "Input" field and a "Submit" button. The "Input" field contains the text "receive_deploy_decision".

Screenshots

The screenshot displays the Studio interface, which is divided into two main panels. The left panel shows a workflow graph for an agent named 'agent'. The graph consists of several nodes connected by arrows: 'intent_decomposer' (red) points to 'ask_human' (yellow), which points to 'technical_mapper' (green), then 'ask_dataset' (green), 'receive_dataset' (purple), 'data_engineer' (yellow), 'ml_dev' (pink), 'logic_dev' (green), and finally 'integrator' (blue). There are also dashed feedback loops from 'ask_human' back to 'intent_decomposer' and from 'ml_dev' back to 'data_engineer'. The right panel shows a thread view for the 'resource_scheduler_optimizer' thread, specifically 'TURN 1'. It displays a tree structure of data under the 'data_engineer' node, including 'Data_Paths' (with values for historical_training_data_path, profiler_notes, streaming_mock_data_path, test_data_path, test_label_column, and training_data_profile), 'Intent_Blueprint', 'Technical_Mapping' (with Control_Service_Model and Reporting_Service_Model), 'Action_Space_Menu', and 'Telemetry_Variables'. Below this, there is a 'messages [...]' section. At the bottom of the interface, there is an 'Input' field with up and down arrows, a 'Manage Assistants' button, a 'As Node' dropdown menu currently set to 'receive_deploy_decision', and a 'Submit' button. The top right of the interface shows 'Connected' status and buttons for 'Interact', 'Trace', and 'Run experiment'.

Studio / agent Graph Chat Connected

Memory Interrupts 3

intent_decomposer

ask_human

technical_mapper

ask_dataset

receive_dataset

data_engineer

ml_dev

logic_dev

integrator

Input ↑ ↓ >

Manage Assistants As Node receive_deploy_decision Submit

Thread resource_scheduler_optimizer + Interact Trace Run experiment

TURN 1

- data_engineer
 - blueprint
 - Data_Paths
 - historical_training_data_path data/historical_training_data.csv
 - profiler_notes All required synthetic datasets were generated successful...
 - streaming_mock_data_path data/streaming_mock_data.json
 - test_data_path data/test_data.csv
 - test_label_column high_load
 - training_data_profile supervised_labeled
 - Intent_Blueprint
 - Technical_Mapping
 - Control_Service_Model MAC
 - Reporting_Service_Model MAC
 - Action_Space_Menu
 - Telemetry_Variables
 - messages [...]

58 minutes ago

- ml_dev
 - blueprint
 - Data_Paths
 - Intent_Blueprint
 - ML_Model_Artifacts
 - Technical_Mapping

Work Distribution

Name	Core Responsibility	Key Tasks & Contributions
Mohit Patil	Design, Planning and Execution	Modules 1-7 Scaffoldings and Initial implementation, Tools developments and expansion and Langchain Deployment, Logic Developer (Module 5), Integrator Development (Module 6), Deployer (Module 7)
Harshwardhan	Design and Execution	FlexRIC Structural RAG, Mapper (Module 2), Data Synthesizer (Module 3) and ML Module (Module 4) with Feature Updates, Threshold and Recursive Training (Module 4), Integrator template designing (Module 6) Documentation and README

Conclusions

Summary of Key Contributions

- Open-source, modular framework inspired by AutORAN
- Intent-driven xApp development
- Blueprint pipeline with shared structured state
- Ensures consistency and traceability
- Built on FlexRIC, extensible to other RICs
- Uses code-aware RAG to map NL → real APIs
- Generates deployable outputs

Impact of the Work

- Lowers barrier to xApp development for non-experts
- Converts high-level intents into validated, deployable apps
- Modular design aligned with real engineering roles
- Intuitive, maintainable, and explainable architecture
- Shared, user-validated blueprint reduces LLM hallucinations
- Enables reliable and scalable network app automation

Future Work

- Develop a deployment and monitoring module that can deploy xApps in a live environment, continuously analyze runtime logs, and autonomously detect and correct issues.
- Integrate state-of-the-art LLM APIs to improve reasoning quality, robustness, and overall system performance.
- Package the pipeline as a fully deployable end-to-end application, moving beyond a LangChain/LangGraph prototype to a production-ready system.
- Enhance the RAG subsystem to reduce hallucinations and make it codebase-agnostic, enabling support for multiple RAN frameworks beyond FlexRIC.
- Evaluate the system on more complex and diverse intents to uncover edge cases, latent bugs and failure modes.
- Introduce a formal code verification stage (e.g. SonarQube-style analysis) after Modules 3–6 to ensure code quality, correctness and security.

References

- GitHub Repo Link:

<https://github.com/mohitpatil30501/Intent-Driven-Autonomous-ORAN-xApp-Development-using-Agentic-AI>

- OAI [<https://gitlab.eurecom.fr/oai/openairinterface5g>]

- Flexric [<https://gitlab.eurecom.fr/mosaic5g/flexric>]

- Langchain

[[\">https://academy.langchain.com/courses/foundation-introduction-to-langchain-python](https://academy.langchain.com/courses/foundation-introduction-to-langchain-python)]

- Ollama [<https://docs.ollama.com/>]

- Intent-Driven Network Management with Multi-Agent LLMs: The Confucius Framework

[<https://dl.acm.org/doi/10.1145/3718958.3750537>]

- AutORAN: LLM-driven Natural Language Programming for Agile xApp Development

[<https://arxiv.org/pdf/2603.18604>]

Declaration on LLM Usage

- List which LLMs you leveraged
 - Gemma4:31B, Claude Anthropic, Gemini 3.1 PRO, ChatGPT
- Include some sample prompts you used
 - Build 5 different well defined User intents - Monitoring, orchestration, optimization, anomaly detection
 - How to evaluate structural RAGs
- How did you validate LLM generated scripts?
 - By Executing on smaller chunk of data
- How did you document your LLM generated code snippets?
 - GitHub Tagging and The dynamic JSON schema natively documents its own reasoning via the blocks